



```
alembic@goatmire:~$ mix ash.demystify --city varberg
```

Ash Demystified

Supercharge your Elixir apps with Ash

event Goatmire Elixir 2026 where Varberg, Sweden when 2026-09-28 13:00 by Alembic

```
$ cat schedule.txt
```

13:00	Resources, actions, relationships	30 min
-------	-----------------------------------	--------

13:30	🔧 Hands-on · labs 0 to 3	30 min
-------	--------------------------	--------

14:00	Calculations, aggregates, policies, code interfaces	30 min
-------	---	--------

14:30	🔧 Hands-on · labs 4 to 7	30 min
-------	--------------------------	--------

15:00	☕ Tea and coffee break	15 min
-------	------------------------	--------

15:15	Ash ecosystem	15 min
-------	---------------	--------

15:30	🔧 Choose your own adventure	75 min
-------	-----------------------------	--------

16:45	Q&A · finish!	15 min
-------	---------------	--------

What is Ash?

Ash is an **opinionated, declarative application framework** that brings the **batteries-included** experience to Elixir.

At its heart, Ash models your domain as **Resources** and the **Actions** you can take on them.

Ash is an **opinionated**, declarative application framework that brings the batteries-included experience to Elixir.

One place for each rule

Phoenix + Ecto

LiveView / controller	auth? filters?
-----------------------	----------------

Context	auth? validation? filters?
---------	----------------------------

Changeset	validation?
-----------	-------------

Ecto schema	fields
-------------	--------

Repo / Ecto.Query	filters? tenancy?
-------------------	-------------------

Phoenix + Ash

LiveView / controller	calls the domain
-----------------------	------------------

Domain	code interface
--------	----------------

Resource	fields · actions · validations policies · filters
----------	--

Data layer	AshPostgres
------------	-------------

Ash is an opinionated, **declarative** application framework that brings the batteries-included experience to Elixir.

Say what, not how

lib/blog.ex · Phoenix context

```
def publish_post(user, post) do
  if user.admin? or post.author_id == user.id do
    post
    ▷ Post.changeset(%{
      status: :published,
      published_at: DateTime.utc_now()
    })
    ▷ validate_required([:title, :body])
    ▷ Repo.update()
  else
    {:error, :forbidden}
  end
end
```

lib/blog/post.ex · Ash resource

```
update :publish do
  change set_attribute(:status, :published)
  change set_attribute(:published_at, &DateTime.utc_now/0)
  validate present([:title, :body])
end

policy action(:publish) do
  authorize_if relates_to_actor_via(:author)
  authorize_if actor_attribute_equals(:admin?, true)
end
```

all behaviour is driven by explicit, static declarations

Ash is an opinionated, declarative **application framework** that brings the batteries-included experience to Elixir.

Your domain, not your web layer

- › Ash is not a replacement for Phoenix. It is the **application layer** that Phoenix calls into.
- › It models your domain and **how it can be interacted with**, as actions like `:publish_post` or `:approve_order`.
- › The same actions serve a LiveView, a JSON API, a Mix task or a background job.
- › Actions are **typed** and **introspectable**, so other code can find them and build on them.

note: Ash is not an ORM. Storing data is a feature you add on top of actions.

Ash is an opinionated, declarative application framework that brings the **batteries-included** experience to Elixir.

Batteries included

built in · primitives in Ash itself

- › Validations and changes
- › Policies (authorisation)
- › Filtering, sorting and pagination
- › Calculations and aggregates
- › Data layers: Postgres, SQLite, ETS
- › Multitenancy
- › Code interfaces and notifiers

note: Model your domain, derive the rest.

extensions · read your resources and derive

- › Auth: [AshAuthentication](#)
- › REST: [AshJsonApi](#)
- › GraphQL: [AshGraphQL](#)
- › TypeScript clients: [AshTypescript](#)
- › LLM tools and MCP servers: [AshAi](#)
- › Admin UI: [AshAdmin](#)
- › Background jobs: [AshOban](#)

Three ways to follow along

1 • Run it Clone the repo and start `iex -S mix`. Every example on these slides runs in that shell. Try them while I talk.

2 • Livebook The same examples as a notebook, if you would rather not clone anything. `livebook/goatmire_2026.livemd`.

3 • Read later The slides are a PDF in the repo. Nothing today is lost if you just watch.

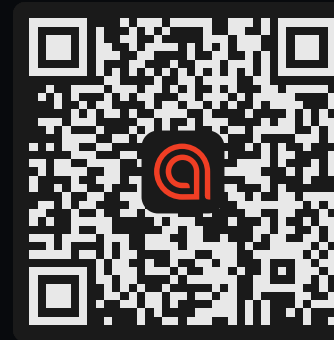
note: Labs come in two blocks. You do not need to keep up with the examples to do them.

PREP

Get the app running

```
git clone https://github.com/team-alembic/ash_training
cd ash_training
git switch lab-00-resources
mise install      # or: asdf install
mise run db-up    # Postgres + pgvector in Docker
mix setup
iex -S mix
```

No Docker? See the README for a local Postgres. Try the examples in `iex` while we talk.



[team-alembic/ash_training](https://github.com/team-alembic/ash_training)

```
$ cd 02_resources
```

02

Resources/

actions.ex attributes.ex relationships.ex data_layer.ex

What is a Resource?

Model your domain, and derive the rest

 *with Resources*

- › the primary concept in Ash
- › usually a **domain object** in your system
- › a **noun** or **entity** in your domain model (eg **User**, **Order**, **Product**)

Resources define the following

- | | | |
|----|----------------------|---|
| 01 | actions | the verbs or commands for your Resources |
| 02 | attributes | the data fields (with validations) |
| 03 | relationships | the relationships between Resources |
| 04 | data_layer | how Ash will persist data (by default it doesn't) |

note: Resources define much, much more that we'll cover later

Resource example

A minimal `Profile` resource with a `UUID` primary key, a `string` attribute `name`, and `create` and `read` actions.

```
lib/account/profile.ex

defmodule Account.Profile do
  use Ash.Resource, domain: Account

  actions do
    read :read
    create :create do
      accept [:name]
    end
  end

  attributes do
    uuid_primary_key :id
    attribute :name, :string
  end
end
```

Domain

Domains are **groupings of related resources** with shared configuration.

You can think of them like a **Phoenix Context**, a **DDD Bounded Context** or a **Service**.

Twitter.Accounts

User · Token ·
OAuthClient

Twitter.Tweets

Tweet · Like

Twitter.Chat

Conversation · Message

Twitter.Agents

McpActions

the domains in today's demo app

Domain example

Resources must also be defined in a Domain.

lib/account.ex

```
defmodule Account do
  use Ash.Domain

  resources do
    resource Account.Profile
  end
end
```

Actions are the foundation of a Resource

- › Ash Resources model both data and operations (data access and transformation)
- › This a fundamental difference between Ash and Ecto
- › All access to a Resource's data is done via an Action
 - This allows Ash to consistently apply rules like validation, authorisation, etc
 - Ash also knows who is calling each action (the actor), which policies build on later

Remember: A Resource without Actions is useless!

Mandatory Attributes

By default `name` is optional. How can we enforce it?

```
attribute :name, :string do  
  allow_nil? false  
end
```

Keyword List vs Block syntax

block · this is called block syntax, and is usually preferred

```
attribute :name, :string do
  allow_nil? false
end
```

keyword list · can also be written inline using keyword list syntax

```
attribute :name, :string, allow_nil?: false
```

both compile to the same attribute entity

Constraining Attribute values

We want to add an attribute called `status` to our profile.

- > type `:atom`
- > can only be the values `:published` or `:unpublished`
- > defaults to `:unpublished`
- > cannot be `nil`

Published Attribute

```
attribute :status, :atom do
  allow_nil? false
  constraints [one_of: [:unpublished, :published]]
  default :unpublished
end
```

Timestamps

Ash provides these to keep track of when the Resource was first created, and when it was last updated:

```
timestamps()
```

> [create_timestamp](#)

> [update_timestamp](#)

Generates `inserted_at` and `updated_at`. For different names (our Profile uses `created_at`), write it long-hand:

```
create_timestamp :created_at  
update_timestamp :updated_at
```

```
$ cd 02_resources/data_layers
```

02.1

Data Layers/

ash_postgres ash_sqlite ash_csv ets mnesia

What is a Data Layer?

By default Ash does not persist any data.

An Ash Data Layer specifies where your Resource's data will be stored.

Which Data Layer?

- › AshPostgres
- › AshSqlite
- › AshCsv
- › Ets
- › Mnesia

More to come!

💡 You should generally default to **AshPostgres**

Specifying a Data Layer

lib/account/profile.ex

```
defmodule Account.Profile do
  use Ash.Resource,
    domain: Account,
    data_layer: AshPostgres.DataLayer

  postgres do
    table "profiles"
    repo Account.Repo
  end

  ...
end
```

Ash Generators

Composable, source patching generators powered by **Igniter**.

```
mix ash.gen.resource      # Generate and configure an Ash.Resource.
mix ash.gen.domain        # Generates an Ash.Domain
mix ash.gen.change        # Generates a custom change module.
mix ash.gen.custom_expression # Generates a custom expression module.
mix ash.gen.enum          # Generates an Ash.Type.Enum
mix ash.gen.preparation   # Generates a custom preparation module.
mix ash.gen.validation    # Generates a custom validation module.
mix ash.extend            # Adds an extension to a resource (e.g. postgres)
mix ash.set.domains       # Scans the app and updates `ash_domains` config
```

More info at [🔗 https://github.com/ash-project/igniter](https://github.com/ash-project/igniter)

```
> mix ash.gen.resource Twitter.Tweets.Tweet
```

```
Igniter:
```

```
Update: lib/twitter/tweets.ex
```

```
... |  
12 12 |  
13 13 |   resources do  
    14 + |     resource Twitter.Tweets.Tweet  
14 15 |   end  
15 16 |end  
... |
```

```
Create: lib/twitter/tweets/tweet.ex
```

```
1 | defmodule Twitter.Tweets.Tweet do  
2 |   use Ash.Resource,  
3 |     otp_app: :twitter,  
4 |     domain: Twitter.Tweets  
5 | end  
6 |
```

```
Proceed with changes? [Yn] y
```

```
$ cd 03_actions
```

03

Actions/

create.ex read.ex update.ex destroy.ex generic.ex

What is an Action?

- › An operation or command that can be performed on a Resource
- › 5 main types **Create, Read, Update, Destroy** or **Generic**
- › **Can and should be named using Domain language where possible**
- › Actions can also be defined on the Domain using Code Interfaces
- › Public by default — `public? false` keeps an action out of API extensions (GraphQL, JSON:API, ...)

Use Domain Language (Not just CRUD)

Actions should be named using **Domain language** where possible. For example:

- > `publish` an `Article` resource making it publicly visible
- > `like` or `unlike` a `Tweet` by updating the `User` / `Tweet` relationship
- > `archive` an `Article` by soft-deleting it

These are really `update` action types.

Default Actions Example

Actions are defined on a **Resource** in an `actions` block

If you just need CRUD actions, the defaults are all you need:

```
actions do
  defaults [
    :read,
    :destroy,
    create: [:name],
    update: [:name]
  ]
end
```

Accepted Attributes

Create and Update **Actions** must explicitly **accept** attributes they will set

```
actions do
  create :create do
    accept [:name]
  end
end
```

Profile Resource

```
lib/account/profile.ex

defmodule Account.Profile do
  use Ash.Resource, domain: Account

  actions do
    read :read
    create :create do
      accept [:name]
    end
  end

  attributes do
    uuid_primary_key :id
    attribute :name, :string, allow_nil?: false
    attribute :status, :atom do
      allow_nil? false
      constraints [one_of: [:unpublished, :published]]
      default :unpublished
    end

    create_timestamp :created_at
    update_timestamp :updated_at
  end
end
```

Calling `create` Actions

To call a create Action, we use `Ash.Changeset.for_create/3` to create a changeset. Then we call `Ash.create!()`

```
Account.Profile
```

```
▷ Ash.Changeset.for_create(:create, %{name: "My Name"})
```

```
▷ Ash.create!()
```

```
#⇒ %Account.Profile{name: "My Name", ...}
```

Note: There are nicer ways to call actions, that we'll get to later on

Action Errors

Now if we create a Profile we should see an error:

```
Account.Profile
▷ Ash.Changeset.for_create(:create, %{})
▷ Ash.create!()
```

produces the following:

```
** (Ash.Error.Invalid) Invalid Error
* attribute name is required
```

Read All Profiles

Now, read all the generated Profiles.

```
Account.Profile
```

```
▷ Ash.read!()
```

```
#=> [  
  %Account.Profile{...},  
  %Account.Profile{...},  
  %Account.Profile{...},  
  %Account.Profile{...}  
]
```

Filtered Reads

Fetching the "Joe Armstrong" Profile requires a filter:

```
require Ash.Query
```

```
Account.Profile
```

```
▷ Ash.Query.filter(name = "Joe Armstrong")
```

```
▷ Ash.read!()
```

```
#⇒ [%Account.Profile{name: "Joe Armstrong", ...}]
```

Sorting and Limits

What if we want to get the latest Profile?

We set our `created_at` timestamp now we can sort on it.

```
Account.Profile
▷ Ash.Query.sort(created_at: :desc)
▷ Ash.Query.limit(1)
▷ Ash.read_one!()
```

```
#⇒ %Account.Profile{...}
```

Calling `update` Actions

With the default `update` action accepting `:name`, build a changeset from an existing record:

```
profile
```

```
▷ Ash.Changeset.for_update(:update, %{name: "Robert Virding"})
```

```
▷ Ash.update!()
```

```
#⇒ %Account.Profile{name: "Robert Virding", ...}
```

Calling **destroy** Actions

Destroying works the same way:

```
profile
  ▷ Ash.Changeset.for_destroy(:destroy)
  ▷ Ash.destroy!()

#⇒ :ok
```

Generic Actions

Generic actions accept input arguments, execute custom logic, and return a value.

Minimal structural constraints, useful for arbitrary business logic. A generic action is structured using `Ash.ActionInput` to manage inputs and `Ash.run_action` for execution.

```
action :say_hello, :string do
  argument :name, :string, allow_nil?: false
  run fn input, _ →
    {:ok, "Hello: #{input.arguments.name}"}
  end
end
```

Running Generic Actions

Build an `Ash.ActionInput`, then run it:

```
Account.Profile
```

```
▷ Ash.ActionInput.for_action(:say_hello, %{name: "Joe"})
```

```
▷ Ash.run_action!()
```

```
#⇒ "Hello: Joe"
```

Builtin Validations

Actions can validate their input with `validate` and a rich set of builtins.

Limiting tweet length:

```
create :create do
  accept [:text]
  validate present(:text)
  validate byte_size(:text, max: 280)
end
```

Many more builtins: `string_length`, `attribute_in`, `compare`, ...

The **actor**

The **actor** is whoever performs an action. It is not a BEAM process. For now, think of it as the current user. We come back to it in Policies.

The hooks on a query/changeset/input may need to know the actor, so set it when you build them:

Post

```
▷ Ash.Query.for_read(:read, actor: current_user)
▷ Ash.read!()
```

```
$ cd 04_relationships
```

04

ReLationshIps/

belongs_to has_one has_many many_to_many

Relationships

Relationships describe the connections between resources, they enable:

- › Loading related data
- › Filtering on related data
- › Managing related records through changes on a single resource
- › Authorizing based on the state of related data

Relationship Basics

A Relationship exists between a source resource and a destination resource. They are defined in the `relationships` block of the source resource.

```
lib/content/post.ex

defmodule Content.Post do
  use Ash.Resource

  attributes do
    uuid_primary_key :id
    attribute :title, :string
  end

  relationships do
    belongs_to :author, Account.Profile
  end
end
```

Kinds of Relationships

There are 4 kinds of relationships:

- > belongs_to
- > has_one
- > has_many
- > many_to_many

Belongs To

`belongs_to` links a `source_attribute` to a `destination_attribute` on another Resource

```
# on Content.Post
belongs_to :author, Account.Profile
```

The source attribute on `Content.Post` is `:author_id` and the destination attribute on `Account.Profile` is `:id`.

Has One

`has_one` is similar to a `belongs_to` except the reference attribute is on the destination resource, instead of the source.

```
# on Account.Profile
has_one :avatar, Account.Avatar
```

The source attribute on `Account.Profile` is `:id` and the destination attribute on `Account.Avatar` is `:profile_id`. This expects that `profile_id` is unique on Avatar.

Has Many

`has_many` relationship is similar to a `has_one` except that the destination attribute is not unique, and will produce a list of related items.

```
# on Account.Profile
has_many :posts, Content.Post
```

The `source_attribute` on `Account.Profile` is `:id` and the `destination_attribute` on `Content.Post` is `:profile_id`.

`has_many` also accepts `sort` and `limit`, e.g. "the 3 most recent posts".

Many To Many

A `many_to_many` relationship can be used to relate many source resources to many destination resources.

To achieve this, the `source_attribute` and `destination_attribute` are defined on a **join resource**.

A `many_to_many` relationship can be thought of as a combination of a `has_many` relationship on the source/destination resources and a `belongs_to` relationship on the join resource.

Through Relationships

Relationships can traverse *other* relationships, using a **path of relationship names**:

```
# on Twitter.Tweets.Tweet
has_many :likers, Twitter.Accounts.User do
  through [:likes, :user]
end
```

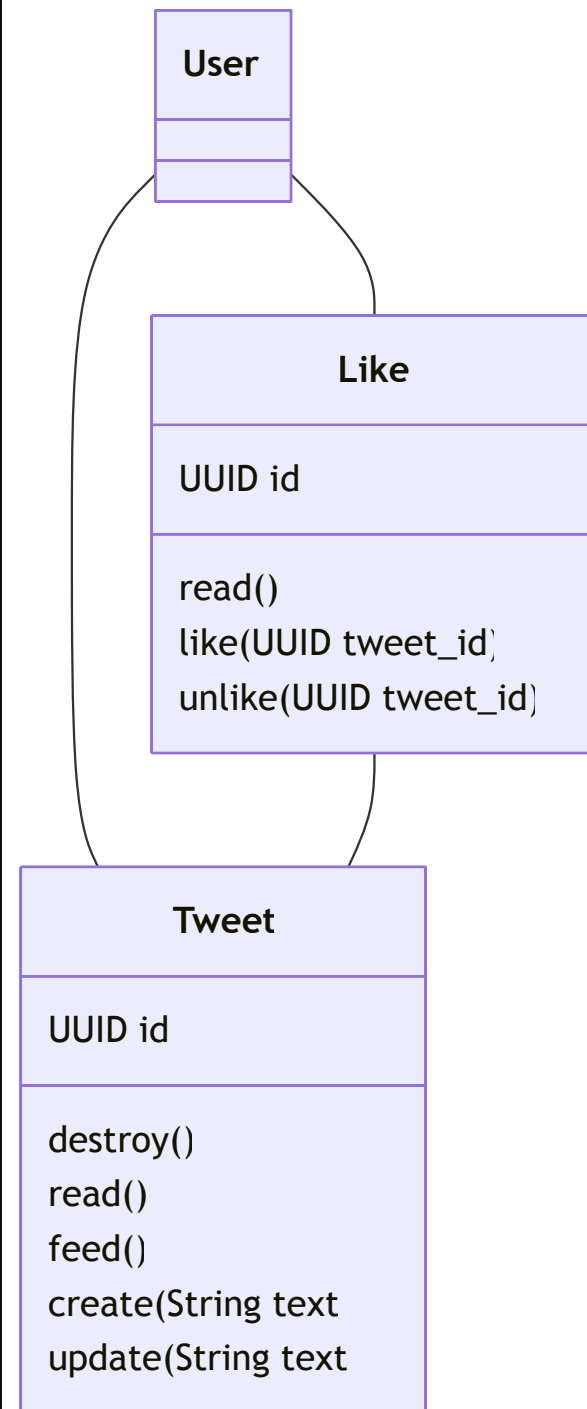
Tweet `has_many :likes`, each like `belongs_to :user` — so `:likers` goes straight from a tweet to the users who liked it.

No join resource or extra columns needed!

Mermaid Resource Diagrams

Ash draws a class diagram of every resource and relationship in your domains.

```
mix ash.generate_resource_diagrams
```



```
$ cd 05_advanced_actions
```

05

Advanced Actions/

```
prepare.ex change.ex validate.ex
```

Custom Read Actions with `prepare`

For `read` Actions, you can add custom behavior with `prepare`.

Let's create a meaningful `latest` Action which sorts by the most recently created Profiles.

```
read :latest do
  prepare build(sort: [created_at: :desc])
end
```

Remember: **actions should be meaningful in your domain, not just CRUD.**

Running the Actions

```
Account.Profile
```

```
▷ Ash.Query.for_read(:latest)
```

```
▷ Ash.read!()
```

```
#⇒ [
```

```
  %Account.Profile{...},
```

```
  %Account.Profile{...}
```

```
]
```

Builtin vs Custom `prepare`s

`build` is a builtin `prepare` function (or "preparation")

```
prepare build(sort: [created_at: :desc])
```

but we can call our own code by providing a Module

```
prepare Account.Profile.Preparations.SortByMostRecentlyCreated
```

Custom `prepare` Module

To define a Preparation we `use Ash.Resource.Preparation` and define a `prepare/3` function.

lib/account/profile/preparations/sort_by_most_recently_created.ex

```
defmodule Account.Profile.Preparations.SortByMostRecentlyCreated do
  use Ash.Resource.Preparation

  @impl true
  def prepare(query, _, _) do
    Ash.Query.build(query, sort: [created_at: :desc])
  end
end
```

Custom Actions with `change` and `validate`

For `create`, `update` and `destroy` Actions, you can add custom behavior with `change` and validations with `validate`.

Let's create a meaningful `publish` Action for Profile which changes the `status`.

```
update :publish do
  # We don't want to accept any input here
  accept []

  change set_attribute(:status, :published)
  validate string_length(:name, min: 2, max: 255)
end
```

Running the Actions

The `destroy` example earlier removed the seeded `profile`, so fetch a fresh one first:

```
profile = Account.Profile ▷ Ash.Query.filter(name = "Joe Armstrong") ▷ Ash.read_one!()
```

```
profile
```

```
▷ Ash.Changeset.for_update(:publish)
```

```
▷ Ash.update!()
```

Builtin vs Custom `change`

`set_attribute` is a builtin `change`

```
change set_attribute(:status, :published)
```

but we can call our own code by providing a Module

```
change Account.Profile.Changes.Publish
```

Custom `change` Module

To define a change we `use Ash.Resource.Change` and define a `change/3` function.

lib/account/profile/changes/publish.ex

```
defmodule Account.Profile.Changes.Publish do
  use Ash.Resource.Change

  @impl true
  def change(changeset, _, _) do
    Ash.Changeset.force_change_attribute(changeset, :status, :published)
  end
end
```

Builtin vs Custom `validate`

`string_length` is a builtin validation

```
validate string_length(:name, min: 2, max: 255)
```

but we can call our own code by providing a Module

```
validate Account.Profile.Validations.CheckNameLength
```

Custom `validate` Module

To define a validation we `use Ash.Resource.Validation` and define a `validate/3` function.

lib/account/profile/validations/check_name_length.ex

```
defmodule Account.Profile.Validations.CheckNameLength do
  use Ash.Resource.Validation

  @impl true
  def validate(changeset, _, _) do
    name = Ash.Changeset.get_attribute(changeset, :name)
    length = String.length(name)

    if length ≥ 2 and length ≤ 255 do
      :ok
    else
      {:error, field: :name, message: "must be at least 2 characters and less than 255"}
    end
  end
end
```

Validations are simpler than changes

Key difference between a validation vs change

- › a validation cannot return a new changeset

```
$ cd 05_advanced_actions/identities
```

05.1

Identities/

identity.ex upserts.ex

What is an Identity?

Identities declare that a record can be uniquely identified by some attributes.

```
identities do
  identity <name>, <keys>
  ...
end
```

Similar idea but not the same as a unique constraint because Ash isn't constrained by a specific data layer

How Does Ash Handle Identities?

Allows fields to be passed to `Ash.get/3`

```
Ash.get!(Account.Profile, %{name: "Joe Armstrong"})
```

AshPostgres creates unique constraints in the database automatically for each identity.

Upserts

Ash automatically handles upserting on primary key, but you need to specify upsert behaviour for other attributes.

```
create :create_or_publish do
  accept [:name]
  change set_attribute(:status, :published)
  upsert? true
  upsert_identity :profile_name
end
```

Upserts are create actions

Did it create or update? The returned record tells you which branch was taken:

```
Ash.Resource.get_metadata(record, :upsert_action)  
#⇒ :insert / :update
```

Expressions with `expr`

Ash expressions give you a way to define **portable calculations** and filters.

This means that they are data layer independent!

```
expr(first_name <> " " <> last_name)

change filter(expr(tweet_id = ^arg(:tweet_id) and user_id = ^actor(:id)))
```

This can be run in Elixir *or* within a data layer for efficient sorting and filtering. `^arg/1` and `^actor/1` pin in the action's arguments and the actor.

HANDS-ON · PART 1

Your turn

```
git switch lab-00-resources           # Lab 0 · Resources
git switch lab-01-basic-actions-and-attributes # Lab 1 · Basic actions
git switch lab-02-relationships        # Lab 2 · Relationships
git switch lab-03-advanced-actions     # Lab 3 · Advanced actions
```

- Each branch has its instructions in `labs/`, e.g. `labs/0-resources.md`.
- Stuck? Switch to the next branch. It starts from the solved lab before it.

```
$ cd 06_calculations_aggregates
```

06

Calculations and Aggregates/

calculations.ex expr.ex aggregates.ex

Calculations

Calculations are derived fields. They can reference attributes, calculations and aggregates.

lib/resource.ex

```
defmodule Resource do
  ...

  calculations do
    calculate <name>, <type>, expr(<expression>)
  end
end
```

Split `name` in the `Person` resource

lib/account/person.ex

```
defmodule Account.Person do
  use Ash.Resource, domain: Account

  actions do
    defaults [:read]

    create :create do
      accept [:first_name, :last_name]
    end
  end

  attributes do
    uuid_primary_key :id
    attribute :first_name, :string, allow_nil?: false
    attribute :last_name, :string, allow_nil?: false
  end
end
```

Calculate `full_name`

Add the `calculations` section to the Resource

```
calculations do
  calculate :full_name, :string, expr(first_name < " " < last_name)

  # or let Ash infer the type from the expression
  calculate :full_name, :auto, expr(first_name < " " < last_name)

  # or reference a Module
  calculate :full_name, :string, Account.Person.Calculations.FullName
end
```

`full_name` can now be loaded on demand or used in filters, sorts, or other calculations.

note: `:auto` works for expression calculations only. Module calculations need an explicit type.

Module Calculations

Not every calculation can be created with an expression.

lib/account/person/calculations/full_name.ex

```
defmodule Account.Person.Calculations.FullName do
  use Ash.Resource.Calculation

  @impl true
  def load(_, _, _), do: [:first_name, :last_name]

  @impl true
  def calculate(records, _, _) do
    Enum.map(records, fn record →
      record.first_name <> " " <> record.last_name
    end)
  end
end
```

Expression Examples

Account.Person

▷ Ash.Query.filter(full_name = "Joe Armstrong")

▷ Ash.read!()

SELECT *

FROM people

WHERE (first_name || ' ' || last_name) = 'Joe Armstrong'

Expression Examples

```
person = %Account.Person{  
  first_name: "Joe",  
  last_name: "Armstrong"  
}
```

```
# calculate in memory from previously defined full_name calculation  
Ash.calculate!(person, :full_name)
```

```
#=> "Joe Armstrong"
```

Calculation Example (Not Loaded)

Let's put that together

```
Account.Person
```

```
▷ Ash.Changeset.for_create(:create, %{first_name: "Joe", last_name: "Armstrong"})
```

```
▷ Ash.create!()
```

```
#⇒ %Account.Person{full_name: #Ash.NotLoaded<:calculation, field: :full_name>, ...}
```

Need to **load** calculations.

Calculation Example (With Load)

Let's put that together

```
Account.Person
```

```
▷ Ash.Changeset.for_create(:create, %{first_name: "Joe", last_name: "Armstrong"})
```

```
▷ Ash.create!()
```

```
▷ Ash.load!([:full_name])
```

```
#⇒ %Account.Person{full_name: "Joe Armstrong", ...}
```

Loading the `full_name` calculates the field, and concatenates the attributes.

Aggregates

🔗 [Aggregates](#) in Ash allow for retrieving summary information over groups of related data.

Some examples:

- › count of published Posts for a User
- › sum of all read counts across all Posts for a User
- › total count of likes for a tweet

Aggregate Example

Given a user `Profile` resource with related `Post` s:

```
aggregates do
  count :count_of_posts, :posts do
    filter expr(published = true)
  end
end
```

Aggregate Types

- | | | |
|----|---------------|---|
| 01 | count | counts related items meeting the criteria |
| 02 | exists | checks if any related items meet the criteria |
| 03 | sum | sums a <code>field</code> of the related items |
| 04 | list | lists a <code>field</code> of the related items |
- # note: ... and more: `first`, `max`, `min`, `avg` and `custom`. See the aggregates guide.

Aggregates can reference Aggregates

The `field` of an aggregate can itself be an aggregate on the related resource.

`like_count` is an aggregate on `Tweet`, so on `User`:

```
sum :total_likes_received, :tweets, :like_count
```

Very little in your database is out of reach once you start stacking aggregates — and it's still a single query.

Unrelated Aggregates

Unrelated aggregates (also called **resource-based aggregates**) allow you to aggregate data from any resource without requiring a predefined relationship.

```
lib/account/user.ex

defmodule Account.User do
  aggregates do
    count :total_profiles,
          Account.Profile

    count :duplicate_name_count,
          Account.Profile do
      filter expr(
        name = parent(name) and
        id ≠ parent(id)
      )
    end
  end
end
```

Inline Aggregates

Aggregates can be written straight into a calculation expression, with no `aggregates` entry.

lib/account/profile.ex

```
calculations do
  calculate :published_post_count, :integer,
    expr(count(posts, query: [filter: expr(published = true)]))

  calculate :post_stats, :map, expr(%{
    published: count(posts, query: [filter: expr(published = true)]),
    total: count(posts)
  })
end
```

note: resource-based aggregates work inline too: `count(Account.Profile, filter: expr(name = parent(name)))`

```
$ cd 07_policies
```

07

Policies/

authorizers.ex policies.ex checks.ex field_policies.ex

Authorization

Authorization in Ash involves three things:

- › `actor` - the entity (i.e User, Org, Device) performing an action, as seen in Actions
- › `authorize?` - a flag that tells Ash to run authorization.
- › `authorizers` - the extensions on a resource that can modify or forbid the action.

Setting `actor` and `authorize?`

All functions in Ash that may perform authorization and/or wish to use the actor accept an `actor` and an `authorize?` option.

```
Ash.Changeset.for_create(  
  Post,  
  :create,  
  %{title: "Post Title"},  
  actor: current_user  
)
```

Authorization runs by default, so there is no need to pass `authorize?: true`.

authorize?: false

Skipping authorization is fine in tests and seeds. In application code it is an anti-pattern. Use a system actor instead, so the call stays traceable, for example in AshPaperTrail's version history.

```
# tests and seeds
```

```
Ash.read!(Post, authorize?: false)
```

```
# application code
```

```
Ash.read!(Post, actor: %MyApp.SystemActor{name: :nightly_digest})
```

Authorizers

Authorizers are in control of what happens during authorization.

Generally, you won't need to create your own authorizer, as the builtin policy authorizer `Ash.Policy.Authorizer` works well for any use case.

```
use Ash.Resource, authorizers: [Ash.Policy.Authorizer]
```

💡 If you don't add at least one Authorizer, your Resource allows any actor to call any action.

A `policies` block without `Ash.Policy.Authorizer` in `authorizers:` fails to compile.

Then you can start defining policies for your resource.

Policies

Policies determine what actions on a resource are permitted for a given actor.

They can also filter the results of read actions to restrict the results to only records that should be visible.

Policy Example

All policies that apply must pass and at least one policy must apply or it's forbidden.

```

policies do
  policy always() do
    authorize_if always()
  end

  policy action_type(:create) do
    authorize_if IsSuperUser
    forbid_if Deactivated
    authorize_if IsAdminUser
    authorize_if HasCreatorRole
  end
end
```

Anatomy of a Policy

Each Policy defined in a resource has two parts:

- 01 a condition, or a list of conditions such as `action_type(:read)` or `always()`. If the condition(s) are true for an attempted action, then the policy will be applied to the action.
- 02 a set of **Checks**, each of which will be evaluated individually if a Policy applies to the attempted action.

How a Policy is processed

If more than one policy applies to any given attempted action (eg. an admin actor calls a read action) then **all applicable policies must pass** for the action to be performed.

A Policy will evaluate to either:

- > `:forbidden`
- > `:authorized`

Policy checks

If no check produces a result then the Policy result is `:forbidden`

> `authorize_if`

- if `true` the whole policy is `:authorized`
- else move to next check

> `authorize_unless`

- if `false` the whole policy is `:authorized`
- else move to next check

Policy checks

If no check produces a result then the Policy result is `:forbidden`

> `forbid_if`

- if `true` the whole policy is `:forbidden`
- else move to next check

> `forbid_unless`

- if `false` the whole policy is `:forbidden`
- else move to next check

Policy Example

```

policies do
  policy action_type(:create) do
    authorize_if IsSuperUser
    forbid_if Deactivated
    authorize_if IsAdminUser
    authorize_if HasCreatorRole
  end
end
```

We check those from top to bottom, so the first one of those that returns `:authorized` or `:forbidden` determines the entire outcome.

A policy for one action

`action_type/1` matches every action of a type. `action/1` matches actions by name.

```

policies do
  policy action_type(:read) do
    authorize_if always()
  end

  policy action(:destroy) do
    authorize_if expr(user_id = ^actor(:id))
  end

  policy action(:create) do
    authorize_if actor_present()
  end
end
```

Custom Error Messages

Policies can carry an `error_message`, used when that policy is the one responsible for the `Forbidden` error.

```
policy action([:update, :destroy]) do
  error_message "only the author can edit or delete a tweet"
  authorize_if expr(user_id == ^actor(:id))
end
```

Much friendlier than the default!

Bypass Policies

A bypass Policy is just like a regular policy, except if a bypass passes, then other policies after it *do not need to pass*.

This can be useful for writing complex access rules, or for a simple rule like "an admin can do anything" without needing to specify it as part of every other policy.

Bypass Example

```
polices do
  bypass IsSuperUser do
    authorize_if always()
  end
end
```

Policy Breakdown Example

Policy Breakdown

Admins and managers can create posts | :

authorize if: actor.admin = true | x | ↓

authorize if: actor.manager = true | x | ↓

Filter Policies

On a read action, a policy check that can be expressed as a filter narrows the results instead of forbidding the whole read.

```
policy action_type(:read) do
  authorize_if expr(published = true)
  authorize_if expr(author_id = ^actor(:id))
end
```

The actor gets back published posts and their own drafts. Other records are filtered out with no `Forbidden` error.

note: set `access_type :strict` on a policy to forbid instead of filter

Checking Permissions (without acting)

Ask the policies a question — no action is run:

```
Ash.can?({tweet, :destroy}, current_user)
#⇒ true | false

# batch multiple checks, sharing the actor
Ash.can_do_all?([{tweet, :update}, {tweet, :destroy}], current_user)
```

Great for showing/hiding buttons in the UI.

Field Policies

Hide a single field instead of the whole record:

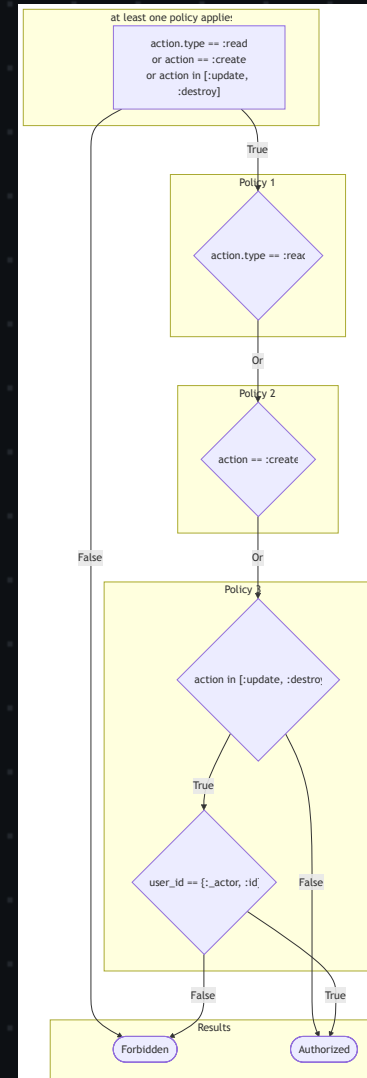
```
field_policies do
  field_policy :email do
    authorize_if expr(id = ^actor(:id))
  end

  field_policy :* do
    authorize_if always()
  end
end
```

Forbidden fields load as `%Ash.ForbiddenField{}`.

Mermaid Policy Flowcharts

`mix ash.generate_policy_charts` draws a flowchart of every policy on a resource.



```
$ cd 08_code_interfaces
```



Code Interfaces/

```
define.ex  default_options.ex  can.ex
```

Why do we need Code Interfaces?

Using `Changeset`s and `Query`s directly to act on resources is a bit unwieldy.

Code Interfaces simplify how we use our defined actions, and offer a clean and rich interface to our Domain.

They can be defined on the **Domain**

Code Interface Example

In this example, we will define it on the **Domain**.

lib/account.ex

```
defmodule Account do
  use Ash.Domain

  resources do
    resource Account.Profile do
      define :create_profile, args: [:name], action: :create
    end
  end
end
```

Using Code Interfaces

Create a profile with 1 line. (Remember we defined it on the Account domain)

```
Account.create_profile!("José Valim")
```

instead of 3 lines

```
Account.Profile  
▷ Ash.Changeset.for_create(:create, %{name: "José Valim"})  
▷ Ash.create!()
```

Nice! the bang version of the function is created as well as the normal version

Default Options

An interface can carry its own defaults with `default_options` — call-site options are merged over them.

```
define :feed,  
  default_options: [load: [:like_count, :liked_by_me]]
```

Call sites shrink to:

```
Twitter.Tweets.feed!(actor: current_user)
```

Predicate Interfaces

A name ending in `?` generates a single function returning a **bare boolean** (raising on errors).

```
define :liked?, args: [:tweet_id]
```

```
Twitter.Tweets.liked?(tweet_id, actor: user)  #⇒ true / false  
Twitter.Tweets.liked(tweet_id, actor: user)   #⇒ {:ok, boolean}
```

`can_*` Helpers

Every `define` also generates `can_*` / `can_*?` helpers that check your policies **without running the action**.

```
Twitter.Tweets.can_delete_tweet?(current_user, tweet)
#⇒ true / false
```

Handy for hiding the delete button on tweets the user can't delete.

HANDS-ON · PART 2 · THEN A 15 MIN BREAK

Your turn

```
git switch lab-04-calculations    # Lab 4 · Calculations
git switch lab-05-aggregates     # Lab 5 · Aggregates
git switch lab-06-policies       # Lab 6 · Policies
git switch lab-07-code-interface # Lab 7 · Code interfaces
```

→ Still on labs 0 to 3? Finish those first, or switch ahead to catch up.



sleep(900_000)

break time! • back in 15 minutes

```
$ cd 09_ecosystem
```



Ecosystem Overview/

data_layers apis web finance utilities admin ai

The Ash Ecosystem

The Ash ecosystem consists of numerous hex packages, all of which have their own documentation.

If you can't find something in this documentation, don't forget to search in any potentially relevant package.

Data Layers

01	AshPostgres	PostgreSQL data layer
02	AshSqlite	SQLite data layer
03	AshCsv	CSV data layer
04	AshCubdb	CubDB data layer

Community Data Layers

01	AshNeo4j	Neo4j graph database
02	AshAge	Apache AGE graph database on Postgres
03	AshScylla	ScyllaDB and Apache Cassandra
04	AshClickhouse	ClickHouse
05	AshDynamo	DynamoDB

API Extensions

01	AshJsonApi	JSON:API builder
----	------------	------------------

02	AshGraphQL	GraphQL builder
----	------------	-----------------

Web

- | | | |
|----|---------------------------------|---|
| 01 | AshPhoenix | Phoenix integrations |
| 02 | AshAuthentication | Authenticate users with password, OAuth, and more |
| 03 | AshAuthenticationPhoenix | Integrations for AshAuthentication and Phoenix |

Finance

- | | | |
|----|-----------------------|---|
| 01 | AshMoney | A money data type for Ash |
| 02 | AshDoubleEntry | A double entry system backed by Ash Resources |

Resource Utilities

- | | | |
|----|------------------------|--|
| 01 | AshOban | Background jobs and scheduled jobs for Ash, backed by Oban |
| 02 | AshArchival | Archive resources instead of deleting them |
| 03 | AshStateMachine | Create state machines for resources |
| 04 | AshPaperTrail | Keep a history of changes to resources |
| 05 | AshCloak | Automatically encrypt (and decrypt) resource attributes |

Admin & Monitoring

- | | | |
|----|-------------------------|---|
| 01 | AshAdmin | A push-button admin interface |
| 02 | AshAppsignal | Monitor your Ash resources with AppSignal |
| 03 | OpenTelemetryAsh | Open Telemetry for Ash resources |

AI & LLM Integration

- | | | |
|----|---------------|--|
| 01 | AshAi | Comprehensive LLM toolbox with chat, tool calling, vectorization, MCP servers, and prompt-backed actions |
| 02 | AshLua | Lua scripting surface for Ash — pairs with AshAi for agent tooling |

Dispelling Myths and Misconceptions!

- › Ash works with Phoenix, it's not a replacement
- › Ash does more than CRUD
- › The investment is definitely worth it, but hard to tell on toy apps
- › Ash doesn't lock you in, escape hatches everywhere
- › Ash is not just about creating APIs
- › Ash has various test helpers, but you can test apps the way you normally do

Choose your own adventure

- | | | |
|----|-------------------|---|
| 01 | forms | Forms and LiveView with AshPhoenix, module 10 (the default) |
| 02 | apis | JSON:API and GraphQL, module 11 |
| 03 | ai | AshAi and MCP servers, module 12 |
| 04 | extensions | Write your own Ash extension with Spark, module 13 |

note: Pick a track and work through it. The rest of the deck is bonus reading.

```
$ cd 10_phoenix_forms
```

10

Forms and LiveView/

```
# track: forms
```

A form is a view of an action

`Twitter.Tweets.Tweet` already declares everything a form needs.

- › `accept [:text]` says which fields the form has
- › `validate string_length(:text, max: 255)` says when it is invalid
- › `change relate_actor(:user)` says what the actor contributes
- › the policies say who may submit it

`AshPhoenix.Form` reads that declaration. You do not restate it in the web layer.

note: One form per action, not per resource. Create and update are two different forms.

Step 1 • the domain grows a form builder

Add `AshPhoenix` to the domain extensions. Every `define` then gets a matching `form_to_*` function.

lib/twitter/tweets.ex

```
defmodule Twitter.Tweets do
  use Ash.Domain,
    extensions: [AshJsonApi.Domain, AshAdmin.Domain, AshPhoenix]

  resources do
    resource Twitter.Tweets.Tweet do
      define :create_tweet, action: :create
      define :update_tweet, action: :update
    end
  end
end
```

Step 2 • build the form from the code interface

lib/twitter_web/live/tweet_live/form.ex

```
defp assign_form(%{assigns: %{tweet: tweet}} = socket) do
  form =
    if tweet do
      Twitter.Tweets.form_to_update_tweet(tweet,
        as: "tweet",
        actor: socket.assigns.current_user
      )
    else
      Twitter.Tweets.form_to_create_tweet(
        as: "tweet",
        actor: socket.assigns.current_user
      )
    end

  assign(socket, form: to_form(form))
end
```

Step 3 • validate on every keystroke

lib/twitter_web/live/tweet_live/form.ex

```
def handle_event("validate", %{"tweet" => tweet_params}, socket) do
  {:noreply,
   assign(socket, form: AshPhoenix.Form.validate(socket.assigns.form, tweet_params))}
end
```

`validate/2` runs the action's changes and validations without touching the data layer. The returned form carries the errors.

Step 4 • submit runs the action

lib/twitter_web/live/tweet_live/form.ex

```
def handle_event("save", %{"tweet" => tweet_params}, socket) do
  case AshPhoenix.Form.submit(socket.assigns.form, params: tweet_params) do
    {:ok, _tweet} →
      socket =
        socket
        ▷ put_flash(:info, "Tweet #{socket.assigns.form.source.type}d successfully")
        ▷ push_navigate(to: ~p"/")

      {:noreply, socket}

    {:error, form} →
      {:noreply, assign(socket, form: form)}
  end
end
```

note: One `submit/2` for create and update. The form already knows which action it wraps.

Step 5 • the template

lib/twitter_web/live/tweet_live/form.ex

```
<.form for={@form} id="tweet-form" phx-submit="save" phx-change="validate">
  <.input label="Text" type="textarea" field={@form[:text]} />
  <footer>
    <.button phx-disable-with="Saving..." variant="primary">Save Tweet</.button>
    <.button navigate={~p"/}>Cancel</.button>
  </footer>
</.form>
```

`@form[:text]` works because `:text` is in the action's `accept` list. An action argument would render the same way.

What the LiveView no longer owns

- › no changeset construction
- › no `Ash.create` or `Ash.update` branching
- › no error translation
- › no copy of the validation rules

note: Change the action and every form built from it changes with it.

lab_08 · zsh

```
$ git switch lab-08-ash-phoenix-form
```

LAB 8

AshPhoenix.Form in TweetLive.Form

→ Follow `labs/8-ash-phoenix-form.md`

→ Try writing more than 255 characters and watch the error appear on keystroke.

```
$ cd 11_apis
```

11

JSON:API and GraphQL/

```
# track: apis
```

Both APIs read the same actions

An API extension does not add a second model of your domain. It names the actions you already have and exposes them.

- › **AshJsonApi** derives a JSON:API compliant REST API
- › **AshGraphQL** derives an Absinthe schema

The `:feed` read action becomes an index route in one and a `feed` query in the other. Filtering, sorting, and pagination come from the resource.

The actor comes from the pipeline

lib/twitter_web/router.ex

```
import AshAuthentication.Plug.Helpers

pipeline :api do
  plug :accepts, ["json"]
  plug :load_from_bearer
  plug :set_actor, :user
end
```

`load_from_bearer` puts the user on the conn. `set_actor` is what the Ash APIs read. Without it policies and `liked_by_me` run with no actor.

Step 1 • `public?` decides what an API can see

```
attribute :text, :string do
  public? true
end

timestamps public?: true

aggregates do
  count :like_count, :likes do
    public? true
  end
end
```

note: Attributes, calculations, and aggregates are private by default. Neither API shows a private field.

```
$ cd 11_apis/json_api
```

11.1

AshJsonApi/

```
type routes field_names swaggerui
```

Step 2 · routes name actions

```
mix ash.extend Twitter.Tweets.Tweet json_api
```

```
lib/twitter/tweets/tweet.ex
```

```
json_api do
  type "tweet"
  field_names :camelize

  routes do
    base "/tweets"

    index :feed
    get :read, primary?: true
  end
end
```

The domain owns the prefix and the router

lib/twitter/tweets.ex

```
use Ash.Domain,  
  extensions: [AshJsonApi.Domain, AshAdmin.Domain, AshPhoenix]  
  
json_api do  
  prefix "/api/json"  
end
```

lib/twitter_web/json_api_router.ex

```
defmodule TwitterWeb.JsonApiRouter do  
  use AshJsonApi.Router,  
    domains: [Twitter.Tweets],  
    json_schema: "/json_schema",  
    open_api: "/open_api"  
end
```

What you get at `/api/json`

```
localhost:4000/api/json/tweets
localhost:4000/api/json/tweets?fields[tweet]=likeCount
localhost:4000/api/json/swaggerui
localhost:4000/api/json/redoc
```

- > `hide_fields [:updated_at]` hides a public field from this API only
- > `description` on an attribute, action, or resource flows into the docs

```
$ cd 11_apis/graphql
```

11.2

AshGraphQL/

type queries mutations playground

Step 3 • queries name actions too

```
mix ash.extend Twitter.Tweets.Tweet graphql --yes
```

```
lib/twitter/tweets/tweet.ex
```

```
graphql do
  type :tweet

  queries do
    list :feed, :feed
  end
end
```

```
query {
  feed(filter: { likeCount: { greaterThan: 0 } }) {
```

The schema is two empty blocks

lib/twitter_web/graphql_schema.ex

```
defmodule TwitterWeb.GraphQLSchema do
  use Absinthe.Schema

  use AshGraphQL, domains: [Twitter.Tweets]

  query do
  end

  mutation do
  end
end
```

note: Types, inputs, and resolvers are generated from the resources. Plain Absinthe fields can sit alongside them.

Narrowing the filter input

```
graphql do
  type :tweet
  filterable_fields [:text, like_count: [:eq, :greater_than]]
end
```

Browse the schema again. The `TweetFilterInput` type shrinks to those fields and those operators.

Mutations • the like actions

lib/twitter/tweets/like.ex

```
graphql do
  type :like

  mutations do
    create :like_tweet, :like do
      result_name :like
    end

    destroy :unlike_tweet, :unlike do
      identity false
    end
  end
end
```

note: `:like` and `:unlike` use `relate\_actor`. Send a bearer token or the mutation fails.

HANDS-ON · LABS 9 AND 10

Your turn

```
git switch lab-09-ash-json-api      # Lab 9 · AshJsonApi
git switch lab-10-ash-graphql      # Lab 10 · AshGraphQL
```

- Follow `labs/9-ash-json-api.md`, then `labs/10-ash-graphql.md`.
- Lab 10 starts from the solved JSON:API lab. Switch ahead if you are stuck.

```
$ cd 12_ash_ai
```

12

AshAi/

```
# AI integration, the Ash way
```

```
mcp.ex chat.ex tools.ex vectorize.ex rag.ex reactor.ex
```

What is AshAi?

Ash Framework extension for AI/LLM integration.

- › Declarative DSL approach
- › Chat generation with tool calling
- › Vectorization for semantic search
- › MCP servers for IDE integration
- › Prompt-backed actions

Key Benefits

- › Define AI tools using familiar Ash patterns
- › Authorization policies automatically apply to AI tool calls
- › Automatic vectorization of resources
- › Expose actions to LLMs through MCP protocol

Five Core Capabilities

1. Chat Generation

Complete chat interfaces with streaming, persistence, and tool calling

2. Tool Calling

Expose Ash actions as LLM-callable tools

3. Vectorization

Automatic embeddings for semantic search

4. MCP Servers

Model Context Protocol for IDE integration

5. Prompt-backed Actions

Structured LLM outputs with type safety

```
$ cd 12_ash_ai/mcp
```

12.1

MCP Servers/

AshAi.Mcp.Dev AshAi.Mcp.Router ash_lua oauth2

Model Context Protocol (MCP)

Standardized protocol for exposing tools to AI assistants.

Supported by Claude Code, Cursor, Zed, ...

AshAi provides MCP server support.

Development vs Production

Development

- › No authentication
- › Development tools
- › For IDE integration

Production

- › Optional authentication
- › Controlled tool access
- › For end users

Development MCP Setup

Add to your `endpoint.ex` inside the `code_reloading?` block:

```
if code_reloading? do
  # ... existing code ...

  plug AshAi.Mcp.Dev,
    # many AI tools still expect the older protocol version
    protocol_version_statement: "2024-11-05",
    otp_app: :twitter,
    path: "/ash_ai/mcp"
end
```

Now AI coding tools can discover and call your actions at

`http://localhost:4000/ash\_ai/mcp`

Production MCP Setup

Expose only the tools you list — nothing else:

```
# In your router — no special pipeline needed
scope "/api" do
  scope "/mcp" do
    forward "/", AshAi.Mcp.Router,
      tools: [:read_feed],
      protocol_version_statement: "2024-11-05",
      otp_app: :twitter
  end
end
```

Need auth? Add a standard Plug pipeline (e.g. API key) in front.

IDE Integration

Point your AI coding tool at the MCP server:

Claude Code:



```
claude mcp add --transport http ash_ai http://localhost:4000/ash_ai/mcp
```

Zed, Cursor, Windsurf, ... have equivalent config.

AI assistants can now discover and call your Ash actions.

One Tool per Action Doesn't Compose

"How long are the last three tweets?"

With one MCP tool per action:

- › N tool round-trips
- › Intermediate results clog the model's context window
- › The model does the arithmetic in its head

ash_lua: Two Tools Instead of Many

- › `ash_lua_docs` — read documentation for the exposed API surface
- › `ash_lua_eval` — run a Lua script that calls those actions

The model composes the whole task — query + filter + arithmetic — as **one script, one round-trip**.

Pure BEAM Lua runtime (Luerl), no system Lua needed.

Scoping the Surface

AshLua.EvalActions

synthesizes `:docs` and `:eval` actions.

Scripts can **only** call the listed `(resource, action)` pairs.

Register both as ordinary AshAi tools and add them to the MCP `tools:` list.

```
lib/twitter/agents.ex

defmodule Twitter.Agents.McpActions do
  use Ash.Resource,
    domain: Twitter.Agents,
    extensions: [AshLua.EvalActions]

  eval_actions do
    resource Tweet, actions: [:read, :feed, :create]
    resource Like, actions: [:read, :like, :unlike]
    resource User, actions: [:read]
  end
end

defmodule Twitter.Agents do
  use Ash.Domain, otp_app: :twitter, extensions: [AshAi]

  resources do
    resource Twitter.Agents.McpActions
  end

  tools do
    tool :ash_lua_docs, Twitter.Agents.McpActions, :docs
    tool :ash_lua_eval, Twitter.Agents.McpActions, :eval
  end
end
```

ash_lua Security Story

- › Actor, tenant, and context are threaded through to **every** Ash call a script makes
- › Scripts can't escalate, switch tenants, or call actions outside the scoped set
- › All your policies still apply
- › Run multiple agent resources side by side, each with its own scope

lab_11 · zsh

```
$ git switch lab-11-ash-ai-mcp
```

LAB 11

MCP Servers & ash_lua

→ Follow `labs/11-ash-ai-mcp.md`

Securing MCP with OAuth 2.1

Remote MCP clients (claude.ai connectors) authenticate with **OAuth 2.1**:

- › discovery via `401` + `WWW-Authenticate` -> `/.well-known/*` metadata
- › **dynamic client registration** — Claude registers itself
- › authorization code + **PKCE**, audience-bound JWTs
- › your existing sign-in page, plus a consent screen

ash_authentication_oauth2_server

Turns your AshAuthentication setup into an authorization server.

```
mix ash_authentication_oauth2_server.install \
  --accounts Twitter.Accounts --user Twitter.Accounts.User \
  --scope mcp
```

Scaffolds client/code/token/consent **resources**, a config module, secrets, and the `/oauth/*` + `/.well-known/*` routes.

Protecting the MCP endpoint

```
pipeline :mcp do
  plug AshAuthentication.Phoenix.Oauth2Server.BearerPlug,
    oauth2_server: Twitter.Oauth2Server
end
```

The verified user becomes the **actor** for every tool call —
your policies apply to AI agents exactly as they do to humans.

lab_15 · zsh

```
$ git switch lab-15-ash-ai-mcp-oauth
```

LAB 15

OAuth-secured MCP for Claude

```
$ cd 12_ash_ai/chat
```

12.2

Chat & Tool Calling/

```
req_llm ash_ai.gen.chat tools AshAi.ToolLoop
```

Talking to Providers: ReqLLM

AshAi talks to LLM providers through **ReqLLM**:

```
# config/dev.exs (and config/runtime.exs for prod)
config :req_llm, openai_api_key: System.get_env("OPENAI_API_KEY")
```

Models are spec strings — `"provider:model-name"`:

> `"openai:gpt-4o"`

> `"anthropic:claude-sonnet-4-5"`

Swapping providers is a one-string change.

Chat Generation

AshAi provides a generator for complete chat features:

```
mix ash_ai.gen.chat --live
```

This creates:

- › **Conversation** and **Message** resources
- › LiveView components for real-time chat
- › Streaming responses via Phoenix PubSub
- › Durable processing with Ash + Oban
- › Built-in tool calling support

Tool Calling

Tools allow LLMs to call Ash actions.

The AI can:

- › Read data from resources
- › Create or update records
- › Execute actions with authorization

Defining Tools

Tools are defined in your **Domain** using a `tools` block:

```
lib/twitter/tweets.ex

defmodule Twitter.Tweets do
  use Ash.Domain,
    extensions: [AshAi]

  resources do
    resource Twitter.Tweets.Tweet
  end

  tools do
    tool :read_feed, Tweet, :feed do
      description ""
      Retrieve the feed of tweets,
      sorted by most recent first
    end

    tool :create_tweet, Tweet, :create do
      description "Create a new tweet"
    end
  end
end
```

Making Attributes Public

For tools to access data, attributes must be marked as `public?`:

```
lib/twitter/tweets/tweet.ex

defmodule Twitter.Tweets.Tweet do
  # ...

  attributes do
    attribute :text, :string do
      allow_nil? false
      public? true # AI can read this
    end

    attribute :internal_notes, :string do
      public? false # AI cannot access
    end
  end
end
```

Tool Calling in Action

User: *"Show me the latest tweets"*

- 01 LLM selects the `read_feed` tool
- 02 Tool executes the `:feed` action
- 03 Results returned to LLM
- 04 LLM responds with formatted answer

Authorization and policies apply to all tool calls.

Running the Agent Loop

`AshAi.ToolLoop` drives the LLM ↔ tools cycle:

```
prompt_messages
▷ AshAi.ToolLoop.stream(
  otp_app: :twitter,
  tools: [:read_feed, :create_tweet], # or `tools: true` for all
  model: "openai:gpt-4o",
  actor: context.actor,
  tenant: context.tenant
)
```

- `AshAi.ToolLoop.run/2` — single result
- `AshAi.ToolLoop.stream/2` — streamed content, tool calls, tool results

The generated chat feature uses this inside a message change.

lab_12 · zsh

```
$ git switch lab-12-ash-ai-chat
```

LAB 12

Chat Setup & Tool Calling

→ Follow `labs/12-ash-ai-chat.md`

```
$ cd 12_ash_ai/vectorization
```

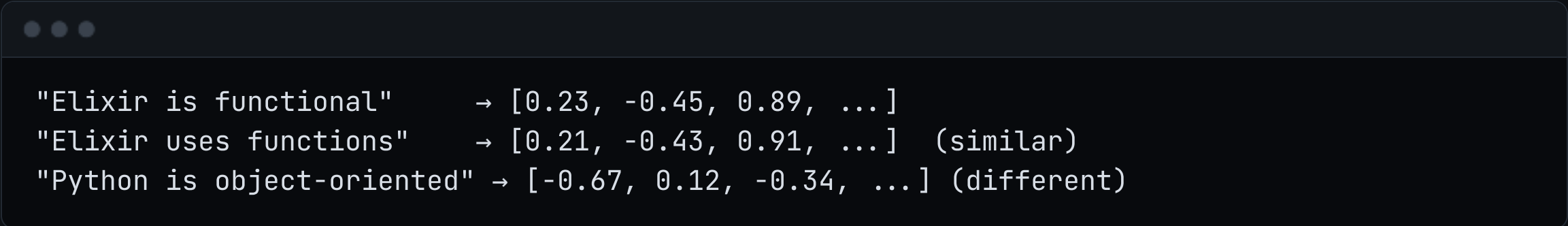
12.3

Vectorization & Semantic Search/

embeddings · vectorize · strategies · semantic_search

Embeddings

Convert text to numerical vectors that capture semantic meaning.

A terminal window with a dark background and light gray text. It contains three lines of text showing sentence embeddings. The first line is "Elixir is functional" followed by an arrow and a vector [0.23, -0.45, 0.89, ...]. The second line is "Elixir uses functions" followed by an arrow and a vector [0.21, -0.43, 0.91, ...] with the word "(similar)" in parentheses. The third line is "Python is object-oriented" followed by an arrow and a vector [-0.67, 0.12, -0.34, ...] with the word "(different)" in parentheses.

```
"Elixir is functional"      → [0.23, -0.45, 0.89, ...]  
"Elixir uses functions"    → [0.21, -0.43, 0.91, ...] (similar)  
"Python is object-oriented" → [-0.67, 0.12, -0.34, ...] (different)
```

Enables semantic search by meaning, not just keywords.

Vectorization in AshAi

Automatic vectorization using the `vectorize` block:

```
lib/twitter/tweets/tweet.ex

defmodule Twitter.Tweets.Tweet do
  use Ash.Resource,
    extensions: [AshAi, AshOban]

  vectorize do
    full_text do
      text fn tweet → "Tweet: #{tweet.text}" end
      used_attributes [:text]
    end

    attributes text: :full_text_vector

    embedding_model {AshAi.EmbeddingModels.ReqLLM,
      model: "openai:text-embedding-3-small", dimensions: 1536}

    strategy :ash_oban
  end
end
```

Vectorization Strategies

AshAi supports three strategies for when to create embeddings:

Strategy	When	Best For
<code>:ash_oban</code>	Background job (async)	Production apps (recommended)
<code>:after_action</code>	Immediately, in-transaction	Small datasets, blocks requests
<code>:manual</code>	You control when	Custom workflows, optimization

Recommendation: Use `:ash_oban` for production - async, resilient, observable

Custom provider? Implement the `AshAi.EmbeddingModel` behaviour (two callbacks) and pass your module as the `embedding_model`.

Semantic Search Action

Create a read action that searches by vector similarity.

```
actions do
  read :semantic_search do
    argument :query, :string, allow_nil?: false

    prepare fn query, _context →
      search_text = Ash.Query.get_argument(query, :query)

      {:ok, [embedding]} =
        AshAi.EmbeddingModels.ReqLLM.generate([search_text],
          model: "openai:text-embedding-3-small",
          dimensions: 1536
        )

      query
      ▷ Ash.Query.sort(
        {calc(vector_cosine_distance(full_text_vector, ^embedding)), :asc}
      )
      ▷ Ash.Query.limit(10)
    end
  end
end
```

Semantic vs Keyword Search

Keyword Search

```
Tweet
▷ Query.filter(
  contains(text, "functional")
)
```

Finds: Exact word matches

Misses: Synonyms, related concepts

Semantic Search

```
Tweet
▷ Query.for_read(
  :semantic_search,
  %{query: "functional"}
)
```

Finds: "functional", "declarative", "pure functions", "immutable"

lab_13 · zsh

```
$ git switch lab-13-ash-ai-vectorization
```

LAB 13

Vectorization & RAG

→ Follow `labs/13-ash-ai-vectorization.md`

```
$ cd 12_ash_ai/rag
```

12.4

RAG (Retrieval-Augmented Generation) /

```
retrieve inject prompt
```

What is RAG?

Retrieval-Augmented Generation enhances LLM responses with relevant context.

Workflow:

- 01 Vectorize user's question
- 02 Retrieve semantically similar content
- 03 Inject context into prompt
- 04 LLM generates answer using context

RAG with Prompt Actions

Retrieve context and generate answers in a single action:

```
action :ask, :string do
  argument :question, :string, allow_nil?: false

  prepare fn input, _context →
    # Retrieve relevant tweets
    context_tweets =
      Tweet
      ▷ Ash.Query.for_read(:semantic_search, %{query: input.arguments.question})
      ▷ Ash.Query.limit(3)
      ▷ Ash.read!()

    Ash.ActionInput.set_argument(input, :context, format_tweets(context_tweets))
  end

  ...
end
```

```
action :ask, :string do
  ...

  run prompt(
    "openai:gpt-4o-mini",
    prompt: ""
    You are answering questions about tweets.

    Relevant tweets:
    <%= @input.arguments.context %>

    Question: <%= @input.arguments.question %>
    ""
  )
end
```

The prompt is an EEx template — arguments set in `prepare` flow straight in.

RAG Example Flow

User: *"What are people saying about Elixir?"*

- 01 Question vectorized → `[0.23, -0.45, ...]`
- 02 Top 3 similar tweets retrieved
- 03 Context injected into prompt
- 04 LLM generates answer using tweet context

```
$ cd 12_ash_ai/reactor
```

12.5

Advanced Orchestration with Reactor/

steps concurrency compensation retries

Why Reactor?

Action hooks have limitations:

- › Sequential execution only
- › No automatic rollback
- › Limited error handling

Reactor provides:

- › Concurrent execution
- › Saga pattern with compensation
- › Automatic retries
- › Built-in telemetry

Reactor vs Action Hooks

Aspect	Action Hooks	Reactor
Execution	Sequential	Concurrent where possible
Error Handling	Manual try/catch	Automatic with compensation
Testing	Test entire action	Test individual steps
Observability	Custom logging	Built-in telemetry
Complexity	Simple workflows	Complex workflows
Reusability	Hard to reuse parts	Composable reactors
Rollback	Manual cleanup	Automatic saga pattern

Reactor RAG Example

Multi-LLM workflow: reformulate → retrieve → answer

lib/twitter/ai/rag_reactor.ex

```
defmodule Twitter.Ai.RagReactor do
  use Reactor, extensions: [Ash.Reactor]

  input :question

  # LLM reformulates the question into a search query
  action :reformulate_query, Tweet, :reformulate_query do
    inputs %{question: input(:question)}
  end

  # Semantic search with the reformulated query
  read :fetch_context_tweets, Tweet, :semantic_search do
    inputs %{query: result(:reformulate_query)}
  end

  ...
end
```

Reactor RAG Example (continued)

Build the context from the retrieved tweets, then answer:

```
lib/twitter/ai/rag_reactor.ex

defmodule Twitter.Ai.RagReactor do
  ...

  step :build_context do
    argument :tweets, result(:fetch_context_tweets)
    run fn %{tweets: tweets}, _ → {:ok, format(tweets)} end
  end

  # LLM answers using the retrieved context
  action :generate_answer, Tweet, :answer_with_context do
    inputs %{question: input(:question), context: result(:build_context)}
  end

  return :generate_answer
end
```

Concurrent Execution

Dependencies are declared via `result(...)` — steps with no dependency between them run concurrently:

```
step :fetch_tweets do
  argument :question, input(:question)
end

step :fetch_user_bio do
  argument :question, input(:question)
end

step :build_prompt do
  # Waits for both concurrent steps to complete
  argument :tweets, result(:fetch_tweets)
  argument :bio, result(:fetch_user_bio)
end
```

Running a Reactor from an Action

Pass the Reactor module directly to `run`:

```
action :ask_with_reactor, :map do
  argument :question, :string, allow_nil?: false

  run Twitter.Ai.RagReactor
end
```

- › Action arguments map to Reactor inputs automatically
- › The action context (actor, tenant) flows into the Reactor

Compensating Actions (Saga Pattern)

If a step fails, Reactor unwinds the steps that already ran:

- › `create` / `update` / `destroy` / `action` steps accept `undo_action` — a compensating action Reactor calls automatically
- › Generic steps implement `compensate` — return `:ok` to accept the failure, `:retry` to try the step again
- › Read-only workflows (like our RAG) have nothing to roll back

Error Handling and Retries

LLM APIs are flaky — rate limits, timeouts. Retry with backoff:

```
step :generate_answer do
  argument :question, input(:question)
  argument :context, result(:build_context)

  max_retries 2

  run fn args, _context →
    Tweet
    ▷ Ash.ActionInput.for_action(:answer_with_context, args)
    ▷ Ash.run_action()
  end

  # :retry tells Reactor to run the step again
  compensate fn _error, _args, _context → :retry end

  # Exponential backoff between attempts: 1s, 2s, 4s...
  backoff fn _error, _args, context →
    round(:math.pow(2, Map.get(context, :current_try, 0)) * 1000)
  end
end
```

When to Use Each Approach

Use Action Hooks When:

- › Simple, linear workflows
- › No parallelization needed
- › Minimal error recovery
- › Quick prototyping

Use Reactor When:

- › Complex multi-step workflows
- › Need parallel execution
- › Require automatic rollback
- › Need detailed observability

lab_14 · zsh

```
$ git switch lab-14-ash-ai-reactor
```

LAB 14

RAG with Reactor

→ Follow `labs/14-ash-ai-reactor.md`

```
$ cd 12_ash_ai/best_practices
```

12.6

Best Practices & Ecosystem/

when_to_use docs

When to Use Each Feature

Feature	Use Case
Chat Generation	Conversational interfaces, customer support
Tool Calling	AI agents that need to read/write your data
MCP Servers	Agent integration, developer tools
ash_lua	Compositional agent tasks in one round-trip
Vectorization	Semantic search, similarity matching
RAG	AI answers grounded in your actual data
Reactor	Complex workflows, parallel processing, rollback

Resources & Documentation

- › **HexDocs:** [↗ hexdocs.pm/ash_ai](https://hexdocs.pm/ash_ai)
- › **GitHub:** [↗ github.com/ash-project/ash_ai](https://github.com/ash-project/ash_ai)
- › **Blog Post:** [↗ Alembic AshAi Guide](#)
- › **MCP Spec:** [↗ spec.modelcontextprotocol.io](https://spec.modelcontextprotocol.io)
- › **AshLua:** [↗ hexdocs.pm/ash_lua](https://hexdocs.pm/ash_lua)
- › **Reactor Docs:** [↗ hexdocs.pm/reactor](https://hexdocs.pm/reactor)

lab_11-15 · zsh

LABS 11-15

AshAi in Practice

→ Labs 11 to 15 run on `lab-11-ash-ai-mcp` through `lab-15-ash-ai-mcp-oauth`

Questions?

```
$ cd 13_spark_dsls
```

13

Spark/

github.com/ash-project/spark

What is a DSL?

A little language that is designed for a very specific purpose.

↪ https://en.wikipedia.org/wiki/Domain-specific_language

↪ <https://martinfowler.com/dsl.html>

↪ <https://martinfowler.com/books/dsl.html>

Why DSLs?

- › Concise Representation
 - Minimal + No extraneous syntax
- › Safer
 - Validation + Fewer options less room for error
- › Expressive power
- › Readable by SMEs
 - Allow contributions from domain experts
 - Better, more powerful and expressive abstractions
- › Declarative (less programming)

Issues with macro based DSLs in Elixir

- › More code, more decisions, more complexity
- › Harder to maintain (it's a special snowflake)
- › Harder to test
- › Non standard
- › Lacks some utilities like formatter help no parens makes DSLs more readable typically
- › Doesn't have LS support for editor hints
- › Not easily extensible
- › Still need to document your DSL

```
$ mix spark.what_if
```

**What if we could build
Elixir DSLs
without writing macros?/**

Why Spark?

- › Easier and safer
 - no need to write macros, less plumbing
- › Standardised structure
 - focus on designing the DSL to solve the business problem
 - don't write your own special snowflake
- › Extensibility
 - other libs can come and extend
- › Autocomplete and hover help via Elixir Language Server
- › Auto docs, no docs for struct data declarations
- › Formatting helpers (no parens for your DSL generated for you)

Spark History

- › Started as part of Ash core
- › Powers all of the Ash ecosystem
- › Battle tested
 - but needs some doc love
- › Split out of Ash core in Aug 2022
 - Was part of Ash core from the beginning

```
$ cd 13_spark_dsls/concepts
```

13.1

Spark Basic Concepts/

Spark.Dsl.Section Spark.Dsl.Entity

Spark.Dsl.Section

- › must have at least one top level section
- › can be hidden with `top_level?: true`
- › a singleton - there can be only one
 - multiple instances are merged together automatically
- › can contain nested section and entities

Spark.Dsl.Entity

- › repeatable list items
- › entities can contain other entities
- › some thoughts about merging with Section, but big impacts on Ash ecosystem

```
$ cd 13_spark_dsls/extension
```

13.2

Building an Extension/

archival.ex · info.ex · transformers/ · verifiers/

Everything in Ash is an extension

```
use Ash.Resource,  
  data_layer: AshPostgres.DataLayer,      # extension  
  authorizers: [Ash.Policy.Authorizer],    # extension  
  extensions: [AshGraphQL.Resource, AshAi] # extensions
```

- › `postgres do ... end`, `graphql do ... end`, `vectorize do ... end` are DSL sections that an extension added
- › Ash core uses the same mechanism
- › Your project can define its own extension, no library needed

What we'll build: a mini AshArchival

Add one line to a resource and get **soft deletes**:

- › an `archived_at` attribute is added
- › every **destroy** becomes an update that stamps `archived_at`
- › every **read** skips archived records

```
use Ash.Resource, extensions: [Twitter.Archival]

archive do
  attribute :deleted_at    # optional, defaults to :archived_at
end
```

Anatomy of an extension

Part	Job
Extension	declares DSL sections + which transformers/verifiers run
Info	introspection: read what the user configured
Transformers	compile time, change the resource
Verifiers	compile time, run last, only check the resource

All four run when the resource module compiles. Nothing runs at request time.

Declaring the DSL

```
defmodule Twitter.Archival do
  @archive %Spark.Dsl.Section{
    name: :archive,
    schema: [attribute: [type: :atom, default: :archived_at]]
  }

  use Spark.Dsl.Extension,
    sections: [@archive],
    transformers: [Twitter.Archival.Transformers.SetupArchival]
end
```

- > `schema` is validated at compile time: a typo is a compile error
- > `describe` and `doc` options become hexdocs and editor hover help

Info modules

```
defmodule Twitter.Archival.Info do
  use Spark.InfoGenerator, extension: Twitter.Archival, sections: [:archive]
end

Twitter.Archival.Info.archive_attribute!(Twitter.Tweets.Tweet)
#⇒ :archived_at
```

- › `Spark.InfoGenerator` writes one function per option
- › Transformers and verifiers call the same functions

Transformers

A transformer receives the resource as data and returns a changed copy.

`SetupArchival` makes three changes:

- 01 **Add** the `archived_at` attribute
- 02 **Rewrite** every destroy action into a soft destroy that sets `archived_at`
- 03 **Filter** every read with `is_nil(archived_at)`

`Ash.Resource.Builder` builds each piece as if you had typed it in the DSL. `AshPostgres` sees the new attribute, and `mix ash.codegen` writes the migration.

Transformers run in order

`defaults [:read, :destroy]` become real actions inside one of Ash's own transformers.

`SetupArchival` must run after it:

```
def after?(Ash.Resource.Transformers.SetPrimaryActions), do: true
def after?(_), do: false
```

Spark sorts transformers by their `before?/1` and `after?/1` callbacks.

Verifiers

A verifier runs after every transformer. It reads the resource and cannot change it.

- › Example: check that `archived_at` is a `utc_datetime_usec`
- › A failed check is a compile error that points at `archive.attribute` in the resource
- › The lab's solution has no verifier. Writing one is a stretch goal.

Verifiers are for "this configuration makes no sense" errors.

What you get for ~80 lines

- › `Twitter.Tweets.Tweet` gains a private `archived_at` attribute
- › `delete_tweet!` archives. The feed, `Ash.get` and relationship loads all skip archived tweets.
- › The real `AshArchival` works the same way. It adds `exclude_read_actions`, `archive_related` and upsert handling.

An extension turns "we always do it this way" into one line of DSL.

lab_16 · zsh

```
$ git switch lab-16-spark-extensions
```

LAB 16

Over to you: build your own extension

→ Soft deletes: follow `labs/16-spark-extensions.md`

Next Steps

Read

- › [🔗 Ash Framework](#), the book by Rebecca Le and Zach Daniel
- › [🔗 hexdocs.pm/ash](#) guides and DSL reference
- › Ash: A Gentle Primer
- › [🔗 alembic.com.au/blog](#)

Practise

- › [🔗 Ash Tutorial](#) Livebook
- › Finish the labs

Ask

- › [🔗 Ash Discord](#)
- › [🔗 elixirforum.com/ash](#)
- › [🔗 ash-hq.org/community](#)

Code with an agent

- › [🔗 usage_rules](#) gives your coding agent each package's own rules

What we didn't cover

- › Custom types
- › Notifiers and PubSub
- › Splode errors
- › Seeds and testing
- › Reactor
- › Custom expressions
- › AshOban
- › Pagination
- › Multitenancy
- › Timeouts
- › Monitoring
- › AshTypescript



```
alembic@goatmire:~$ whois alembic
```

Alembic

A consultancy specialising in the Ash Framework

- › Team upskilling and training
- › Building roadmap features
- › Ash Premium Support

web alembic.com.au

